

COURSE NAME: Theory of Computation COURSE CODE: CSA13

COURSE OUTCOME COVERED

CO3:Construct regular expressions, and use the pumping lemma and closure properties to analyze regular languages. (BL : 3)

Assessment Tool Weightage: 20%

CASE STUDY

Total Marks:50

QUESTIONS: 5 Duration: 1
Hour

Code of Conduct:

*I **E.Gopi Krishna** -(192311347) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

1. Consider a case study in parsing nested structured expressions using a Context-Free Grammar (CFG). Given the grammar $S \rightarrow (L)$

| a L $\rightarrow$

L, S | S,

Analyze how the string ((a,a),(a,a)) can be generated. Demonstrate both leftmost and rightmost derivations for the string and construct the corresponding parse tree to show its hierarchical structure. This helps in understanding syntax analysis in compilers.

ANSWER:

Given CFG

$S \rightarrow (L) \mid a$

$L \rightarrow L,S \mid S$

String: ((a,a),(a,a))

Leftmost Derivation

$S \Rightarrow (L)$
 $\Rightarrow (L,S)$
 $\Rightarrow ((L),S)$
 $\Rightarrow ((L,S),S)$
 $\Rightarrow ((S,S),S)$
 $\Rightarrow ((a,S),S)$
 $\Rightarrow ((a,a),S)$
 $\Rightarrow ((a,a),(L))$
 $\Rightarrow ((a,a),(L,S))$
 $\Rightarrow ((a,a),(S,S))$
 $\Rightarrow ((a,a),(a,S))$
 $\Rightarrow ((a,a),(a,a))$

Rightmost Derivation

$S \Rightarrow (L)$
 $\Rightarrow (L,S)$
 $\Rightarrow (L,(L))$
 $\Rightarrow (L,(L,S))$
 $\Rightarrow (L,(L,a))$
 $\Rightarrow (L,(a,a))$
 $\Rightarrow (L,S)$
 $\Rightarrow ((L),S)$
 $\Rightarrow ((L,S),S)$
 $\Rightarrow ((L,a),S)$
 $\Rightarrow ((a,a),S)$
 $\Rightarrow ((a,a),(a,a))$

Parse Tree

S
/|\
(L)
/\
L , S

/ L,S | /
S | \
\ L,S S
(L) | S \ | |
S |
a
S a a |
/ a
(L)

Result: The string ((a,a),(a,a)) is successfully generated, showing its nested hierarchical structure.

2. Describe the step-by-step process of simplifying this CFG. After simplification, convert the grammar into Greibach Normal Form (GNF) and explain how this transformation improves parsing efficiency

$S \rightarrow XY$

$X \rightarrow$

$YS \mid a Y$

$\rightarrow SX \mid$

b

ANSWER:

1. Simplification and GNF Conversion

Given CFG:

$S \rightarrow XY$

$X \rightarrow YS \mid a$

$Y \rightarrow SX \mid b$

Step 1: Remove useless symbols

All three variables **S, X, and Y** are reachable from the start symbol and can derive terminal strings. Therefore, **no useless symbols** are present.

Step 2: Remove ϵ -productions

There are **no ϵ -productions** in the grammar.

Step 3: Remove unit productions

There are **no unit productions** such as $A \rightarrow B$.

Therefore, the simplified CFG remains:

$$S \rightarrow XY$$

$$X \rightarrow YS \mid a$$

$$Y \rightarrow SX \mid b$$

Step 4: Convert into GNF

In **Greibach Normal Form (GNF)**, every production must start with a terminal:

$$A \rightarrow a\alpha$$

where a is a terminal and α is a sequence of variables.

First substitute X in S:

$$S \rightarrow XY$$

$$\rightarrow YSY \mid aY$$

Now substitute Y:

$$S \rightarrow (SX \mid b)SY \mid a(SX \mid b)$$

After substitution:

$$S \rightarrow SXS \mid bSY \mid aSX \mid ab$$

The production $S \rightarrow SXS$ has **left recursion**, so it is eliminated using a new variable, say R.

The resulting productions are then repeatedly substituted so that every production begins with a **or** b. Thus, the final grammar is expressed in the GNF form:

$$S \rightarrow a\alpha \mid b\alpha$$

$$X \rightarrow a\alpha \mid b\alpha$$

$$Y \rightarrow a\alpha \mid b\alpha$$

where α contains only variables.

Why GNF improves parsing

GNF makes **top-down parsing easier and more efficient** because:

1.

Every production begins with a terminal.

2.

3.

The next input symbol helps the parser choose a production.

4.

5.

It reduces unnecessary backtracking.

6.

7.

It is suitable for predictive and recursive-descent parsing.

8.

9.

Each derivation step consumes an input terminal.

10.

Conclusion: The original CFG is already free from ϵ -productions, unit productions, and useless symbols. Converting it to GNF ensures terminal-leading productions, making syntax analysis and top-down parsing more systematic and efficient.

3. As a language designer, explain how you would classify the grammar type and construct a suitable Context-Free Grammar (CFG) for this language. Further, demonstrate the process of derivations and represent a sample program using a parse tree. Explain how ambiguity can arise in such grammars and how it can be eliminated. mention a few sample strings in the language

ANSWER:

$E \rightarrow E + E \mid$
 $E * E \mid (E) \mid id$

1. CFG Classification, Derivation and Parse Tree

Given grammar:

$E \rightarrow E + E \mid E * E \mid (E) \mid id$

Grammar Classification

This is a **Context-Free Grammar (CFG)** because every production has a single non-terminal on the left side. It generates arithmetic expressions containing **addition, multiplication, and parentheses.** Some strings generated by the grammar are: **identifiers,**

Sample Strings

id
id + id
id * id
(id + id)
id + id * id
(id + id) * id

Derivation

Consider the string:

id+id*id

A derivation is:

E

$\Rightarrow E + E$

$\Rightarrow id + E$

$\Rightarrow id + E * E$

$\Rightarrow id + id * E$

$\Rightarrow id + id * id$

Thus, the grammar generates the expression $id + id * id$.

Parse Tree

E

```

      / | E   E * E |
      |       |
      id      id id
      \
      + E /\
```

Ambiguity

The grammar is **ambiguous** because the same expression can have more than one parse tree.

For example:

id+id*id

can be interpreted as:

$(id + id) * id$

or:

$id + (id * id)$

Therefore, the grammar does not specify the **precedence** of + and *.

Removing Ambiguity

We can rewrite the grammar by giving * higher precedence than +:

$E \rightarrow E + T \mid T$

$T \rightarrow T * F \mid F$

$F \rightarrow (E) \mid id$

Now:

•

* has higher precedence than +.

•

•

+ has lower precedence.

•

•

Parentheses have the highest priority.

•

Therefore:

$\text{id} + \text{id} * \text{id}$

is uniquely interpreted as:

$\text{id} + (\text{id} * \text{id})$

Conclusion

The given grammar is a **CFG but ambiguous**. By separating expressions into E, T, and F, operator precedence and associativity can be defined, eliminating ambiguity and producing a unique parse tree.

4. In NLP where a system uses a Context-Free Grammar (CFG) to parse simple sentences and build parse trees. Analyze whether the generated language satisfies closure properties like union and intersection. CFGs are closed under union but not under intersection, showing both their usefulness and limitations in real-world applications.

ANSWER:

CFG Closure Properties: Union and Intersection

A **(CFG)** is commonly used in NLP to parse sentences and

Context-Free Grammar construct showing their structure.

parse trees grammatical

1. Closure under Union

CFGs are .

closed under union

If L_1 and L_2 are two context-free languages, then:

$L_1 \cup L_2$ is also context-free.

For example:

$L_1 = \{\text{sentences with nouns}\}$

$L_2 = \{\text{sentences with verbs}\}$

Their union can be represented by a new CFG using a new start symbol:

$S \rightarrow S_1 \mid S_2$

Thus, the grammar can generate sentences from either language.

2. Closure under Intersection

CFGs are **not closed under intersection**

1 2

If L_1 and L_2 are context-free languages, their intersection:

$$L_1 \cap L_2$$

is **context-free**.

not necessarily

Therefore, we cannot always construct a CFG that generates exactly the strings common to both languages.

Example

Let:

$$L_1 = \{a^i b^j c^i \mid i, j \geq 0\}$$

$$L_2 = \{a^i b^j c^i \mid i, j \geq 0\}$$

Both are context-free languages. Their intersection is:

$$L_1 \cap L_2 = \{a^n b^n c^n \mid n \geq 0\}$$

This language is **not context-free**.

Conclusion

CFGs are **closed under union but not intersection**. This makes CFGs useful for combining different grammatical structures in NLP, but their inability to handle arbitrary intersections limits their ability to represent some complex language constraints.

5. In a symbolic computation system, a language is defined as $L = \{a^i b^j c^k \mid i = j \text{ or } j = k\}$. Analyze whether this language is context-free. Justify your answer by decomposing the language into simpler components and applying closure properties of context-free languages.

ANSWER:

Given the language

$$L = \{a^i b^j c^k \mid i=j \text{ or } j=k\}$$

We can decompose it into two languages:

$$L = L_1 \cup L_2$$

where

$$L_1 = \{a^i b^i c^k \mid i, k \geq 0\}$$

and

[
 $L_2 = \{a^i b^j c^k \mid i, j \geq 0\}$
]

1. Check (L_1)

[
 $L_1 = \{a^i b^i c^k\}$
]

Here, the number of as must equal the number of bs, while any number of cs is allowed. A CFG for (L_1) is:

[
 $S_1 \rightarrow AB$
]
 [
 $A \rightarrow aAb \mid \epsilon$
]
 [
 $B \rightarrow cB \mid \epsilon$
]

Therefore, (L_1) is **context-free**.

2. Check (L_2)

[
 $L_2 = \{a^i b^j c^j\}$
]

Here, any number of as is allowed, while the number of bs must equal the number of cs. A CFG for (L_2) is:

[
 $S_2 \rightarrow AB$
]
 [
 $A \rightarrow aA \mid \epsilon$
]
 [
 $B \rightarrow bBc \mid \epsilon$
]

Therefore, (L_2) is also **context-free**.

3. Apply closure property

Context-free languages are **closed under union**.

Since

[
 $L = L_1 \cup L_2$
]

and both (L_1) and (L_2) are context-free, their union is also context-free.

Conclusion

[

$$L = \{a^i b^j c^k \mid i=j \text{ or } j=k\}$$
 is Context-Free}

The key reason is that the condition “(i=j) or (j=k)” allows us to split the language into two simpler CFLs, and CFLs are closed under **union**.

RUBRICS FOR CALCULATION Weight

Total

Criteria Conceptu		age		Mar k			
		Q1 (10)	Q2 (10)	Q3 (10)	s (50)		
		Q4 (10)	Q5 (10)				
al Understa nding Accuracy of Answer Application of Knowledg e Comple ness of	Response Clarity & Presentati on	3	3				
				2.5 2	2.5 2	2.5 2	
		2.5 2	2.5 2				12.5 10
	30 25 20				1.5 1	1.5 1	
		1.5 1	1.5 1	1.5 1			7.5
	15 10			3	3	15	5
			3				

Total per Question

10

Marks

/10 /10 /10 /10 /10

/ 50